

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский политехнический
университет»**

Электротехнический факультет

Выпускающая кафедра: Информационные технологии и
автоматизированные системы (ИТАС)

Направление подготовки: 09.03.04 Программная инженерия

ОТЧЕТ

Лабораторная работа №12

«Ассоциативные контейнеры библиотеки STL.»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 15

Выполнил: студент группы РИС-25-2Б

Ильинов Фёдор Михайлович

Принял: Доц. Полякова О. А.

Пермь 2026

ПОСТАНОВКА ЗАДАЧИ

Задача 1.

1. Создать ассоциативный контейнер.
2. Заполнить его элементами стандартного типа (тип указан в варианте).
3. Добавить элементы в соответствии с заданием
4. Удалить элементы в соответствии с заданием.
5. Выполнить задание варианта для полученного контейнера.
6. Выполнение всех заданий оформить в виде глобальных функций.

Задача 2.

1. Создать ассоциативный контейнер.
2. Заполнить его элементами пользовательского типа (тип указан в варианте). Для
пользовательского типа перегрузить необходимые операции.
3. Добавить элементы в соответствии с заданием
4. Удалить элементы в соответствии с заданием.
5. Выполнить задание варианта для полученного контейнера.
6. Выполнение всех заданий оформить в виде глобальных функций.

Задача 3

1. Создать параметризованный класс, используя в качестве контейнера ассоциативный контейнер.
2. Заполнить его элементами.
3. Добавить элементы в соответствии с заданием

4. Удалить элементы в соответствии с заданием.
5. Выполнить задание варианта для полученного контейнера.
6. Выполнение всех заданий оформить в виде методов параметризованного класса.

Задание 3

Найти среднее арифметическое и добавить его в конец контейнера

Задание 4

Найти элементы ключами из заданного диапазона и удалить их из контейнера

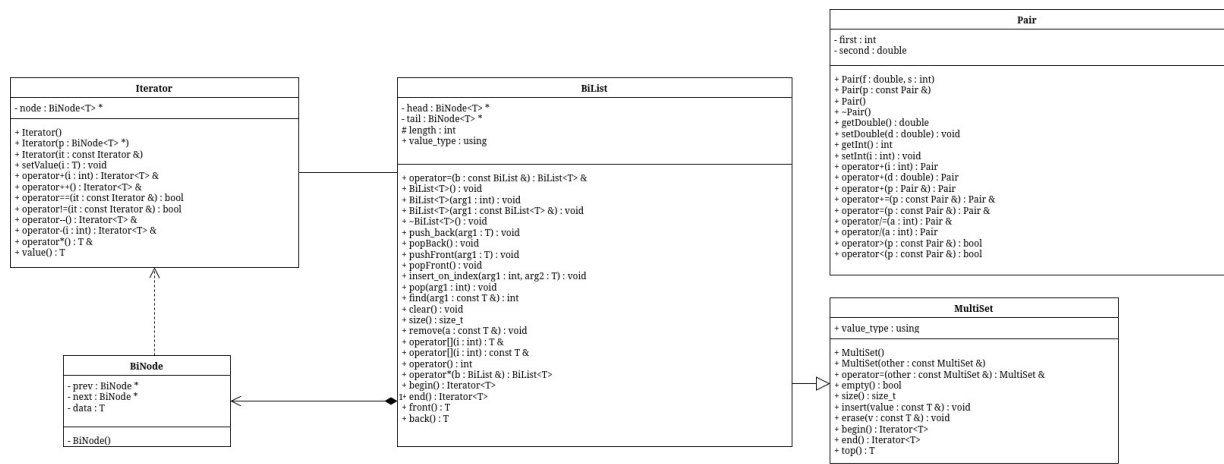
Задание 5

К каждому элементу добавить сумму минимального и максимального элементов контейнера.

АНАЛИЗ.

1. Воспользуемся функциями из работы №11, изменив используемые методы соответствующие для multiset.

UML-ДИАГРАММА



КОД Pair.h

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
#include <ostream>

class Pair {
    int first;
    double second;

public:
    Pair(double, int);
    Pair(const Pair &);
    Pair();
    ~Pair();
    double getDouble() const;
    void setDouble(double);
    int getInt() const;
    void setInt(int);
    Pair &operator=(Pair const &p);
    Pair operator+(Pair &p);
    Pair operator+(int);
    Pair operator+(double);
    Pair& operator+=(Pair const &p);
    Pair& operator/=(int a);
    Pair operator/(int a) const;
    bool operator>(Pair const &p) const;
    bool operator<(Pair const &p) const;
    bool operator==(const Pair &p) const;
    friend std::ostream &operator<<(std::ostream &, const Pair &);
    friend std::istream &operator>>(std::istream &, Pair &);
};

#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_FRACTION_H
```

КОД Pair.cpp

```
#include "Pair.h"
#include <iostream>

Pair::Pair(double a, int b) {
    first = b;
    second = a;
}

Pair::Pair(const Pair &f) {
    second = f.second;
    first = f.first;
}

Pair::Pair() {
    second = 0;
    first = 0;
}

Pair::~Pair() {
    second = 0;
    first = 0;
}

double Pair::getDouble() const {
    return second;
}

void Pair::setDouble(double a) {
    second = a;
}

int Pair::getInt() const {
    return first;
}

void Pair::setInt(int a) {
    first = a;
}

Pair &Pair::operator=(Pair const &p) {
    second = p.second;
    first = p.first;
    return *this;
}
```

```
}
```

```
bool Pair::operator<(Pair const &p) const {  
    if (this->getInt() < p.getInt())  
        return true;  
    if (this->getInt() == p.getInt())  
        return this->getDouble() < p.getDouble();  
    return false;  
}
```

```
bool Pair::operator>(Pair const &p) const {  
    return p < *this;  
}
```

```
Pair & Pair::operator+=(Pair const &p) {  
    this->first += p.first;  
    this->second += p.second;  
    return *this;  
}
```

```
Pair & Pair::operator/=(int a) {  
    this->first /= a;  
    this->second /= a;  
    return *this;  
}
```

```
Pair Pair::operator/(int a) const {  
    return Pair(  
        this->second / a,  
        this->first / a  
    );  
}
```

```
bool Pair::operator==(const Pair &p) const {  
    return (this->first == p.first && this->second == p.second);  
}
```

```
std::ostream &operator<<(std::ostream &stream, const Pair &p) {  
    std::cout << p.first << ":" << p.second;  
    return stream;  
}
```

```
std::istream &operator>>(std::istream &stream, Pair &p) {  
    std::cout << "Double? ";  
    stream >> p.second;  
}
```

```
std::cout << "Int? ";  
stream >> p.first;  
return stream;  
}
```

```
Pair Pair::operator+(Pair &p) {  
    Pair second(*this);  
    second.second += p.second;  
    second.first += p.first;  
    return second;  
}
```

```
Pair Pair::operator+(int a) {  
    Pair tmp(*this);  
    tmp.first += a;  
    return tmp;  
}
```

```
Pair Pair::operator+(double a) {  
    Pair tmp(*this);  
    tmp.second += a;  
    return tmp;  
}
```


KOD MultiSet.hpp

```
#ifndef ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_MultiSet_H
#define ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_MultiSet_H
#include "bidirectional.hpp"

template<class T>
class MultiSet : BiList<T> {
public:
    using value_type = T;

    MultiSet() : BiList<T>() {
    };

    MultiSet(const MultiSet &other) : BiList<T>() {
        for (auto &i: other)
            this->push_back(i);
    };

    MultiSet &operator=(const MultiSet &other) {
        this->clear();
        for (Iterator<T> i = other.begin(); i != other.end(); ++i)
            this->push_back(*i);
        return *this;
    };

    bool empty() const {
        return !this->length;
    };

    size_t size() const {
        return BiList<T>::size();
    };

    void insert(const T &value) {
        int idx = 0;
        for (Iterator<T> i = this->begin(); i != this->end(); ++i, idx++) {
            if (*i < value) {
                this->insert_on_index(idx, value);
                return;
            }
        }
        this->push_back(value);
    }
};
```

```

void erase(const T &v) {
    int i = this->find(v);
    while (i != -1) {
        this->pop(i);
        i = this->find(v);
    }
};

Iterator<T> begin() const {
    return BiList<T>::begin();
}

Iterator<T> end() const {
    return BiList<T>::end();
}

T top() {
    return *BiList<T>::begin();
}
};

```

```
#endif //ILINOV_FEDOR_RIS_25_2B_LABS_PSTU_CS_MultiSet_H
```

КОД main.cpp

```

#include <iostream>
#include <set>
#include <random>
#include "Pair.h"
#include "MultiSet.hpp"

std::random_device rd;
std::mt19937 gen(rd());

template<class T>
void printContainerAfterSubtask(const T& l) {
    std::cout << "\t\t";
    for (auto &i: l) std::cout << i << ' ';
    std::cout << '\n';
}

template<class T>

```

```

T subtask1(const T& t) {
    T l = t;
    std::cout << "\tsubtask 1\n";
    using Item = typename T::value_type;

    Item avg = Item();
    for (auto &i: l)
        avg += i;
    avg /= l.size();
    l.insert(avg);

    std::cout << "\t\t" << "avg: " << avg << '\n';

    return l;
}

```

```

template<class T>
T subtask2(const T& t) {
    T l = t;

    std::cout << "\tsubtask 2\n";
    using Item = typename T::value_type;

    std::uniform_int_distribution<> dist(0, 2);
    std::vector<Item> forDelete;
    for (auto &i: l)
        if (!dist(gen))
            forDelete.push_back(i);

    std::cout << "\t\tFor delete:\n\t\t";
    for (auto &i: forDelete) {
        std::cout << i << ' ';
        l.erase(i);
    }
    std::cout << '\n';

    return l;
}

```

```

template<class T>
T subtask3(const T& t) {
    T l = t;

    std::cout << "\tsubtask 3\n";
    using Item = typename T::value_type;

```

```

    auto mn = *l.begin(), mx = *l.begin();
    for (auto &i: l) {
        if (i < mn) mn = i;
        if (i > mx) mx = i;
    }
    const Item avg = (mn + mx) / 2;

    T tmp;
    for (auto &i: l)
        tmp.insert(i + avg);
    std::cout << "\t\t(min + max) / 2 = " << avg << '\n';

    return tmp;
}

void task_1() {
    std::cout << "task 1\n";

    std::multiset<double> ms;
    std::uniform_int_distribution<> dist(-99, 99);
    for (int i = 0; i < 10; i++)
        ms.insert((double)dist(gen) / 10);

    std::cout << '\t';
    for (auto &i: ms) std::cout << i << ' ';
    std::cout << '\n';

    ms = subtask1(ms);
    printContainerAfterSubtask(ms);

    ms = subtask2(ms);
    printContainerAfterSubtask(ms);

    ms = subtask3(ms);
    printContainerAfterSubtask(ms);
}

void task_2() {
    std::cout << "task 2\n";

    std::multiset<Pair> ms;
    std::uniform_int_distribution<> dist(0, 9);
    for (int i = 0; i < 10; i++) {

```

```

        Pair p((double)dist(gen) / 10, dist(gen));
        ms.insert(p);
    }

    std::cout << '\t';
    for (auto &i: ms) std::cout << i << ' ';
    std::cout << '\n';

    ms = subtask1(ms);
    printContainerAfterSubtask(ms);

    ms = subtask2(ms);
    printContainerAfterSubtask(ms);

    ms = subtask3(ms);
    printContainerAfterSubtask(ms);
}

void task_3() {
    std::cout << "task 3\n";

    MultiSet<Pair> ms;
    std::uniform_int_distribution<> dist(0, 9);
    for (int i = 0; i < 10; i++) {
        Pair p((double)dist(gen) / 10, dist(gen));
        ms.insert(p);
    }

    std::cout << '\t';
    for (auto &i: ms) std::cout << i << ' ';
    std::cout << '\n';

    ms = subtask1(ms);
    printContainerAfterSubtask(ms);

    ms = subtask2(ms);
    printContainerAfterSubtask(ms);

    ms = subtask3(ms);
    printContainerAfterSubtask(ms);
}

int main() {
    task_1();

```

```
task_20;  
task_30;  
}
```

РЕЗУЛЬТАТ ВЫПОЛНЕНИЯ

/home/localuser/CLionProjects/Ilinov_Fedor_RIS_25_2B_Labs_PSTU_CS/sem2/
manual_2_oop/lab12/main

task 1

-7.3 -7 -4.6 -4.3 -2.5 -0.5 -0.1 5.2 7.7 9.4

subtask 1

avg: -0.4

-7.3 -7 -4.6 -4.3 -2.5 -0.5 -0.4 -0.1 5.2 7.7 9.4

subtask 2

For delete:

-4.6 -2.5 -0.1

-7.3 -7 -4.3 -0.5 -0.4 5.2 7.7 9.4

subtask 3

$(\min + \max) / 2 = 1.05$

-6.25 -5.95 -3.25 0.55 0.65 6.25 8.75 10.45

task 2

1:0.1 2:0.1 3:0 3:0.7 6:0.3 6:0.6 7:0.6 8:0.8 9:0.4 9:0.6

subtask 1

avg: 5:0.42

1:0.1 2:0.1 3:0 3:0.7 5:0.42 6:0.3 6:0.6 7:0.6 8:0.8 9:0.4 9:0.6

subtask 2

For delete:

2:0.1 6:0.3 7:0.6 9:0.6

1:0.1 3:0 3:0.7 5:0.42 6:0.6 8:0.8 9:0.4

subtask 3

$$(\min + \max) / 2 = 5:0.25$$

6:0.35 8:0.25 8:0.95 10:0.67 11:0.85 13:1.05 14:0.65

task 3

7:0.1 6:0.8 6:0.6 4:0.5 3:0.8 3:0.7 2:0.8 2:0.3 1:0.8 1:0.2

subtask 1

avg: 3:0.56

7:0.1 6:0.8 6:0.6 4:0.5 3:0.8 3:0.7 3:0.56 2:0.8 2:0.3 1:0.8 1:0.2

subtask 2

For delete:

1:0.2

7:0.1 6:0.8 6:0.6 4:0.5 3:0.8 3:0.7 3:0.56 2:0.8 2:0.3 1:0.8

subtask 3

$$(\min + \max) / 2 = 4:0.45$$

11:0.55 10:1.25 10:1.05 8:0.95 7:1.25 7:1.15 7:1.01 6:1.25 6:0.75 5:1.25

ВОПРОСЫ

1. Что представляет собой ассоциативный контейнер?

Ассоциативный контейнер — это структура, содержащая пары «ключ–значение», где доступ к значению осуществляется по ключу.

2. Перечислить ассоциативные контейнеры библиотеки STL.

map, multimap, set, multiset

3. Каким образом можно получить доступ к элементам ассоциативного контейнера?

Через итераторы и (для map) через операцию []

4. Привести примеры методов, используемых в ассоциативных контейнерах.

insert(), erase(), clear(), find(), count(), lower_bound(), upper_bound(), swap()

5. Каким образом можно создать контейнер map? Привести примеры.

```
map<int, float> m1;
```

```
map<int, float> m2(m1);
```

6. Каким образом упорядочены элементы в контейнере map по умолчанию? Как изменить порядок на обратный?

По возрастанию ключа (less<Key>). Изменить можно через компаратор greater<Key>

7. Какие операции определены для контейнера map?

[], =, ==, !=, <, >, <=, >=, insert(), erase(), clear(), find()

8. Написать функцию для добавления элементов в контейнер map с помощью функции make_pair()

```
template<class T1, class T2>
```

```
void addMap(map<T1, T2>& m, int n) {
```

```
    for(int i = 0; i < n; i++) {
```

```
        T2 val;
```

```
        cin >> val;
```

```
        m.insert(make_pair(i, val));
```

```
    }
```

```
}
```

9. Написать функцию для добавления элементов в контейнер map с помощью операции []

```
template<class T1, class T2>
```

```
void addMapIndex(map<T1, T2>& m, int n) {
```

```
    for(int i = 0; i < n; i++) {
```

```
        T2 val;
```

```
        cin >> val;
```



```

        m[i] = val;
    }
}

```

10. Написать функцию для печати контейнера map с помощью итератора

```

template<class T1, class T2>
void printMap(map<T1, T2>& m) {
    for(auto i = m.begin(); i != m.end(); i++)
        cout << i->first << " " << i->second << '\n';
}

```

11. Написать функцию для печати контейнера map с помощью операции []

```

template<class T1, class T2>
void printMapIndex(map<T1, T2>& m, int n) {
    for(int i = 0; i < n; i++)
        cout << i << " " << m[i] << '\n';
}

```

12. Чем отличаются контейнеры map и multimap?

map не допускает дубликатов ключей, multimap допускает одинаковые ключи

13. Что представляет собой контейнер set?

Контейнер, хранящий уникальные отсортированные элементы без дубликатов

14. Чем отличаются контейнеры map и set?

map хранит пары ключ-значение, set хранит только ключи

15. Каким образом можно создать контейнер set? Привести примеры.

```

set<int> s1;
set<int> s3(s2);

```

16. Каким образом упорядочены элементы в контейнере set по умолчанию? Как изменить порядок на обратный?

По возрастанию (less<T>), изменить на убывание можно через greater<T>

17. Какие операции определены для контейнера set?

insert(), erase(), find(), count(), clear(), begin(), end(), swap()

18. Написать функцию для добавления элементов в контейнер set

```

template<class T>
void addSet(set<T>& s, int n) {
    for(int i = 0; i < n; i++) {

```

```
    T val;  
    cin >> val;  
    s.insert(val);  
}  
}
```